

```

import os
import anthropic
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler, MessageHandler,
    CallbackQueryHandler, ContextTypes, filters
)

# ضع توكناتك هنا
TELEGRAM_TOKEN = "8651997587:AAGdXhtZ0IAsQgRFhkkve_XMiAutC03K2RE"
ANTHROPIC_API_KEY = "sk-ant-api03-SZD_P4XuY_WDWaf3d3wRcjARGPZuY0Gk3qHNW7y1SSeUGZF"

client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)

LANGUAGES = {
    "ar": "🇸🇦 العربية",
    "en": "🇬🇧 English",
    "ru": "🇷🇺 Русский",
    "fr": "🇫🇷 Français",
    "de": "🇩🇪 Deutsch",
    "es": "🇪🇸 Español",
    "it": "🇮🇹 Italiano",
    "tr": "🇹🇷 Türkçe",
    "zh": "🇨🇳 中文",
    "ja": "🇯🇵 日本語",
    "ko": "🇰🇷 한국어",
    "hi": "🇮🇳 हिन्दी",
    "pt": "🇧🇷 Português",
    "fa": "🇮🇷 فارسی",
    "ur": "🇵🇰 اردو",
    "tr": "🇹🇷 Türkçe",
    "nl": "🇳🇱 Nederlands",
    "pl": "🇵🇱 Polski",
    "sv": "🇸🇪 Svenska",
    "uk": "🇺🇦 Українська",
}

# حفظ حالة المستخدم
user_states = {}

def get_lang_keyboard(prefix="tgt"):
    """لوحة اختيار اللغة"""
    keyboard = []
    langs = list(LANGUAGES.items())

```

```

for i in range(0, len(langs), 2):
    row = []
    for code, name in langs[i:i+2]:
        row.append(InlineKeyboardButton(name, callback_data=f"{prefix}:{code}")
    keyboard.append(row)
return InlineKeyboardMarkup(keyboard)

```

```

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    text = f"""

```

🌍 *أهلاً {user.first_name}! في TarjimAI*

.أنا مترجمك الذكي، أترجم بشكل طبيعي وعامي مثل ما يتكلم أهل اللغة

كيف تستخدمني

- 1 أرسل لي أي نص
- 2 اختر اللغة المصدر
- 3 اختر اللغة الهدف
- 4 استلم الترجمة فوراً ✨

الأوامر

/start - البداية

/help - المساعدة

/langs - اللغات المدعومة

*أدعم ٢٠ لغة 🚀

"""

```

    await update.message.reply_text(text, parse_mode="Markdown")

```

```

async def help_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = """

```

? *المساعدة*

الاستخدام البسيط

!فقط أرسل لي النص وأنا أسألك عن اللغات

مثال سريع

`أرسل: كيف حالك؟`

اختر: من العربية ← إلى الروسية

النتيجة: `Как дела?`

مميزاتي

- ✓ ترجمة طبيعية وعامية
- ✓ أفهم السياق
- ✓ ٢٠ لغة
- ✓ سريع وذكي

🎯! أرسل أي نص وابدأ

"""

```
        await update.message.reply_text(text, parse_mode="Markdown")

async def langs_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    langs_text = "🌐 *اللغات المدعومة:*\n\n"
    for code, name in LANGUAGES.items():
        langs_text += f"{name}\n"
    await update.message.reply_text(langs_text, parse_mode="Markdown")

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    text = update.message.text

    # حفظ النص
    user_states[user_id] = {"text": text, "step": "choose_source"}

    await update.message.reply_text(
        f"📝 *النص:*\n`{text[:100]}`{'...' if len(text) > 100 else ''}\n\n📄 الأصلي",
        parse_mode="Markdown",
        reply_markup=get_lang_keyboard("src")
    )

async def handle_callback(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()

    user_id = query.from_user.id
    data = query.data

    if user_id not in user_states:
        await query.edit_message_text("⚠️.انتهت الجلسة، أرسل النص من جديد")
        return

    state = user_states[user_id]

    if data.startswith("src:"):
        lang_code = data.split(":")[1]
        state["source"] = lang_code
        state["step"] = "choose_target"

        src_name = LANGUAGES.get(lang_code, lang_code)
        await query.edit_message_text(
            f"✅ *المصدر:*\n`{src_name}`\n🎯 *اللغة المطلوبة*",
            parse_mode="Markdown",
            reply_markup=get_lang_keyboard("tgt")
        )
```

```

elif data.startswith("tgt:"):
    lang_code = data.split(":")[1]
    state["target"] = lang_code

    src_name = LANGUAGES.get(state["source"], state["source"])
    tgt_name = LANGUAGES.get(lang_code, lang_code)

    await query.edit_message_text(
        f"⌚ *جاري الترجمة...*\n\n{src_name} ← {tgt_name}",
        parse_mode="Markdown"
    )

    # الترجمة
    try:
        translation = await do_translate(
            state["text"],
            src_name,
            tgt_name
        )

        result = f"""
        ✨ *!الترجمة جاهزة*

        *من:* {src_name}
        *إلى:* {tgt_name}

        📖 *الأصل:*
        {state['text']}

        🌐 *الترجمة:*
        {translation}

        _TarjimAI 🤖_
        """

        await query.edit_message_text(result, parse_mode="Markdown")

    except Exception as e:
        await query.edit_message_text(
            "❌ حدث خطأ في الترجمة، حاول مجدداً"
        )

    # تنظيف الحالة
    if user_id in user_states:
        del user_states[user_id]

async def do_translate(text: str, src_lang: str, tgt_lang: str) -> str:

```

```
prompt = f"""{tgt_lang} إلى {src_lang} أنت مترجم محترف. ترجم النص من{src_lang} إلى {tgt_lang}.
```

قواعد مهمة:

- الترجمة تكون طبيعية وعامية كما يتكلمها أهل اللغة
- احتفظ بنبرة الكلام (رسمي/غير رسمي/عاطفي)
- ترجم التعابير الاصطلاحية بمعناها
- أعط الترجمة فقط بدون أي شرح

النص:

```
{text}"""
```

```
response = client.messages.create(  
    model="claude-sonnet-4-20250514",  
    max_tokens=1000,  
    messages=[{"role": "user", "content": prompt}]  
)  
return response.content[0].text.strip()
```

```
def main():
```

```
    app = Application.builder().token(TELEGRAM_TOKEN).build()
```

```
    app.add_handler(CommandHandler("start", start))
```

```
    app.add_handler(CommandHandler("help", help_command))
```

```
    app.add_handler(CommandHandler("langs", langs_command))
```

```
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))
```

```
    app.add_handler(CallbackQueryHandler(handle_callback))
```

```
    print("✅ TarjimAI Bot يشتغل...")
```

```
    app.run_polling(drop_pending_updates=True)
```

```
if __name__ == "__main__":
```

```
    main()
```